

TREG news

Helmut Osterloher

LP DCDC Converters

Agenda

- General improvements
- New features
 - Disable trim step in *.treg file
 - Restart of table char now possible
 - execute() function for TRIM and TRIM_GRP
 - New trim measure group

General improvements

- Under the hood: New list framework with reduced code complexity for easier code maintenance
- No crashes anymore if a parameter is not found (e.g. typo)
- Error messages are reported via the Eagle Message Handler
- TREG checks now if a valid site is passed by the user.
- Statistical trim learning doesn't require "Stop-on-Fail" mode anymore. It now checks the binning information and learns if the unit passed before.
- The order of the definitions in the *.treg file is now arbitrary.

Misc features

- Single trim steps can be disabled in the *.treg file by putting brackets () around the step.
 - Useful e.g. if a step doesn't work (silicon bug)
 - Or one wants to use shorter ramps during characterization
 - The default step (marked with *) is always enabled

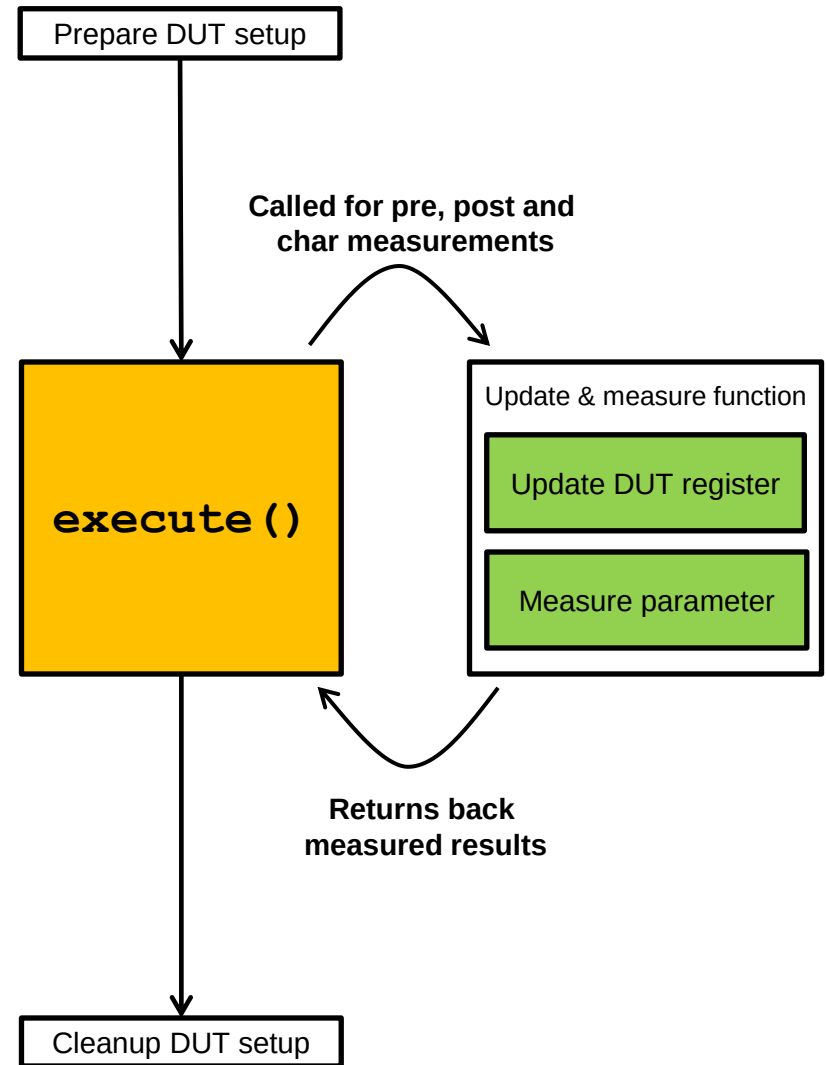
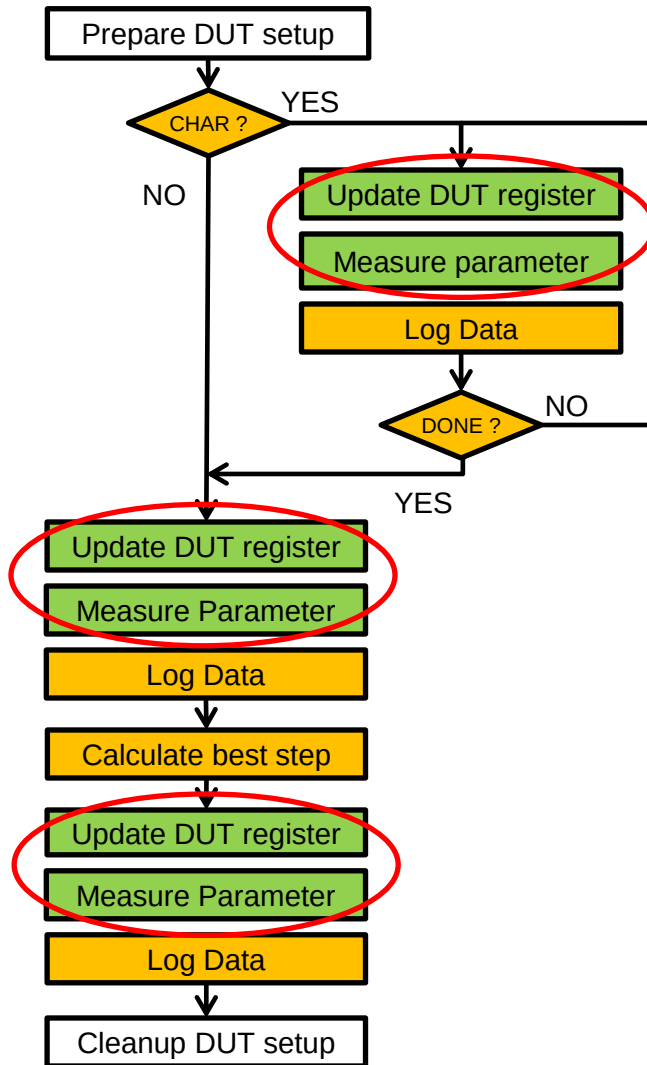
```
[ilim_valley] ; don't use steps 0 and 7  
Table = ( -27.27% ), -18.18%, -9.09%, *0%, 9.09%, 18.18%, 27.27%, ( 36.36% )
```

- Table characterization can be restarted again at any time
 - desirable e.g. on wafer change or every x insertion

```
dut.trim.restart_table_char(); // restart char for all trim parameters  
dut.trim("ICONST").restart_table_char(); // restart char for "ICONST" parameter only
```

```
BOOL Cef3313::OnTestStartup( void ) {  
    if(!(insertion % 200)) // restart char every 200 insertions  
        dut.trim.restart_table_char();  
}
```

execute() function



execute() function

- Execute() ...
 - Implements correct sequence of TREG calls (e.g. pre(), post(),...)
 - Ensures that trim learning works out of the box
 - Eases trim step characterization (for device char and during production)
 - Skips post trim measurement if not necessary (→ saves test time)
 - Takes care of datalogging
 - Test engineer can focus on implementing the measurement

Prerequisites

- For characterization to turn table char always on:

```
dut.force_table_char_active(CHAR_MODE); // CHAR_MODE = true or false
```

- Tell TREG if trimming is enabled or disabled:

```
dut.trim.set_trim_allowed(TRIM_MODE); // TRIM_MODE = true or false
```

Refresher on TREG storages

- Every TRIM parameter or SEL bit field has multiple storages

working

Most important one; Contains data that should be send to the device.
Trimming will update the content of *working* (e.g. during *pre()* or *execute()* calls)

programmed

When **trimming is disabled** the information from *programmed* is copied to *working*
User has to ensure *programmed* matches with the EEPROM information.

start

Stores default value; Changes for TRIM parameter if default learning is enabled.
(For TRIM parameter only readable by user)

read_back

Additional storage. TREG itself doesn't touch or use this.
save_read_back()/restore_readback() can be used to copy data to/from *read_back*

saved

Additional storage. TREG itself doesn't touch or use this.
save_working()/restore_working() can be used to copy data to/from *saved*

Code example

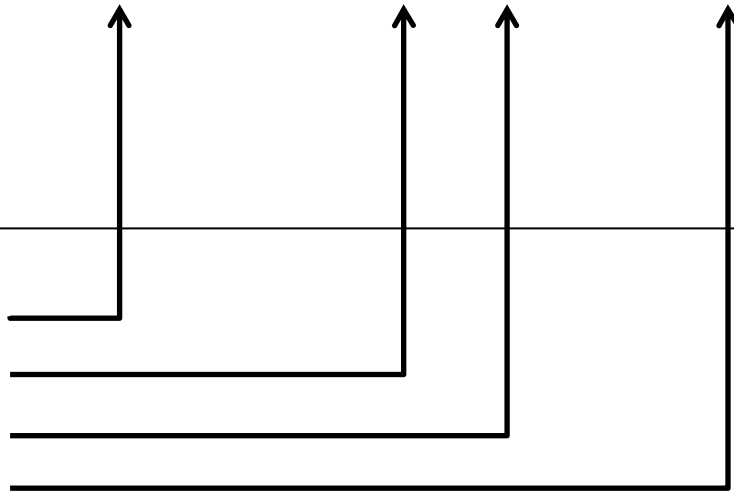
```
// Test Function: iconst_trim
ETS_PRGFLOW_FUNC iconst_trim( int DSIndex, LPCTSTR TestLabel )
{
    // Prepare DUT Setup
    dut.sel("TM").set_working(TM3); // select testmode 3

    dut.trim("iconst").execute( measure iconst trim, 1100, 8200, TREG LOG STD );

    // Cleanup DUT Setup

    return( msSiteStat( MS_ALL ) );
} // END_ETS_PRGFLOW_FUNC
```

Update & measure function
First **production** datalog number
First **characterization** datalog number
Log level (optional)



Update & measure function - Notes

- execute() calls the update & measure function with the 'working' set as needed.
- Update & measure function prototype has to be as following

```
void func_name(TRIM_NODE *trim_node, TREG_MEASURE_FLAG treg_measure_flag, double *results);
```

- trim_node pointer can be used to access TREG internals if needed
- treg_measure_flag is available to determine at which stage the function was called (pre, post or during char)
- results array has to be filled with measurement results

Update & measure function - Example

```
void measure_iconst_trim(TRIM_NODE *trim_node, TREG_MEASURE_FLAG treg_measure_flag, double *results){

    RESULTS_STR iconst_res[NUM_SITES];
    double      en_voltage = 1.0;

    switch(treg_measure_flag){
        case TREG_MEASURE_CHAR: // Special setup for characterization
            break;
        case TREG_MEASURE_PRE:  // Special setup for pre trim measurement
            break;
        case TREG_MEASURE_POST: // Special setup for post trim measurement
            break;
    }

    // Update DUT registers with values from 'working'
    es.write_reg("LAHEAD");

    // Measure parameter
    dpinviset(EN1,DPIN_FV,en_voltage,DPIN_8V,DPIN_8MA);
    dpinviset(EN1,DPIN_FV,en_voltage,DPIN_8V,DPIN_8UA);
    lwait(500); // wait till EN voltage is settled
    dpul6cal.my_dpinmi(iconst_res,EN1,32,13);
    dpinviset(EN1,DPIN_LOAD,spec("ves_pullup"),DPIN_8V,DPIN_512UA);

    // Feedback results to TREG
    SERIAL results[SITE]=-iconst_res[SITE].value*1e-3;
}
```

Datalogging - Standard

- During characterization log all measured steps

8200.1	4.3900	UA	4.0000	7.0000	ICONST_char_	*step_0
8201.1	4.5350	UA	4.0000	7.0000	ICONST_char_	*step_1
8202.1	4.6350	UA	4.0000	7.0000	ICONST_char_	*step_2
...						
8207.1	5.8650	UA	4.0000	7.0000	ICONST_char_	*step_7

- TREG_LOG_TABLE logs in addition trim table during characterization

8208.1	-14.3415	PCNT	-20.0000	20.0000	ICONST_char_	*table_0
8209.1	-11.5122	PCNT	-20.0000	20.0000	ICONST_char_	*table_1
8210.1	-7.7073	PCNT	-20.0000	20.0000	ICONST_char_	*table_2
8211.1	-4.0000	PCNT	-20.0000	20.0000	ICONST_char_	*table_3
8212.1	0.0000	PCNT	-20.0000	20.0000	ICONST_char_	*table_4
8213.1	4.3902	PCNT	-20.0000	20.0000	ICONST_char_	*table_5
8214.1	9.1707	PCNT	-20.0000	20.0000	ICONST_char_	*table_6
8215.1	14.4390	PCNT	-20.0000	20.0000	ICONST_char_	*table_7

- TREG_LOG_STD (=default)

1100.1	4.0000	DB	0.0000	7.0000	ICON_pre_bit	*trim	// log pre trimming step
1101.1	5.1250	UA	4.0000	7.0000	ICON_pre_	*trim	// log pre trimming result
1102.1	3.0000	DB	0.0000	7.0000	ICON_pos_bit	*trim	// log post trimming step
1103.1	1.0000	DB	0.0000	1.0000	ICON_updated	*trim	// log updated_by_trim() flag
1104.1	4.9200	UA	4.0000	7.0000	ICON_theory_	*trim	// log predicted/calculated result
1105.1	5.0000	UA	4.0000	7.0000	ICON_target_	*trim	// log target
1106.1	4.9200	UA	4.6000	5.4000	ICON_pos_	*trim	// log post trimming result

- TREG_LOG_DELTA (=default + deltas)

1107.1	-0.0800	UA	-0.5000	0.5000	ICON_abs_err	*trim	// log absolute delta to target
1108.1	-1.6000	PCNT	-2.0000	2.0000	ICON_rel_err	*trim	// log relative delta to target

Datalogging – Alternative scheme

- To save test#s required for trimming for large test programs there is an alternative datalogging scheme available.
 - Uses **msLogData** for all “non-critical” tests in conjunction with **subtest numbers**
 - Remaining required tests in testlists are:
 - One test# for all trim steps during characterization
 - Optional with TREG_LOG_TABLE: One test# for the trim table
 - One test# for the pre trim result (limits are used for calculated post and target datalogging as well)
 - One test# for the post trim result
 - Optional with TREG_LOG_DELTA: One test# for absolute delta to target
 - Optional with TREG_LOG_DELTA: One test# for relative delta to target
 - Enable alternative datalogging in *.treg file with:

```
[DEFAULT] ; global settings
use_mslogdata = true
```

Datalogging – Alternative scheme: example

- For characterization (limits and units from char test#)

8200.0	4.5823	UA	4.0000	7.0000	iconst_char_step_0
8200.1	4.7336	UA	4.0000	7.0000	iconst_char_step_1
8200.2	4.9372	UA	4.0000	7.0000	iconst_char_step_2
...					
8200.7	6.1219	UA	4.0000	7.0000	iconst_char_step_7

Note: 8200.1 = char test# in testlist

Legend:

- using msLogData
- using testlist
- using msLogData with limits & unit from testlist

- TREG_LOG_TABLE logs in addition trim table during characterization

8201.0	-14.3415	PCNT	-20.0000	20.0000	iconst_char_table_0
8201.1	-11.5122	PCNT	-20.0000	20.0000	iconst_char_table_1
8201.2	-7.7073	PCNT	-20.0000	20.0000	iconst_char_table_2
8201.3	-4.0000	PCNT	-20.0000	20.0000	iconst_char_table_3
8201.4	0.0000	PCNT	-20.0000	20.0000	iconst_char_table_4
8201.5	4.3902	PCNT	-20.0000	20.0000	iconst_char_table_5
8201.6	9.1707	PCNT	-20.0000	20.0000	iconst_char_table_6
8201.7	14.4390	PCNT	-20.0000	20.0000	iconst_char_table_7

Note: 8201.1 = (char test# + 1) in testlist

- TREG_LOG_STD (=default)

1100.0	4.0000	#	0.0000	7.0000	iconst_pre_step
1100.1	5.3495	UA	4.0000	7.0000	iconst_pre_ *value
1100.2	3.0000	#	0.0000	7.0000	iconst_post_step
1100.3	1.0000	#	0.0000	1.0000	iconst_updated
1100.4	5.1355	UA	4.0000	7.0000	iconst_post_calc
1100.5	5.2190	UA	4.0000	7.0000	iconst_target
1101.1	5.1355	UA	4.7000	5.7000	iconst_post_ *value

- TREG_LOG_DELTA (=default + deltas)

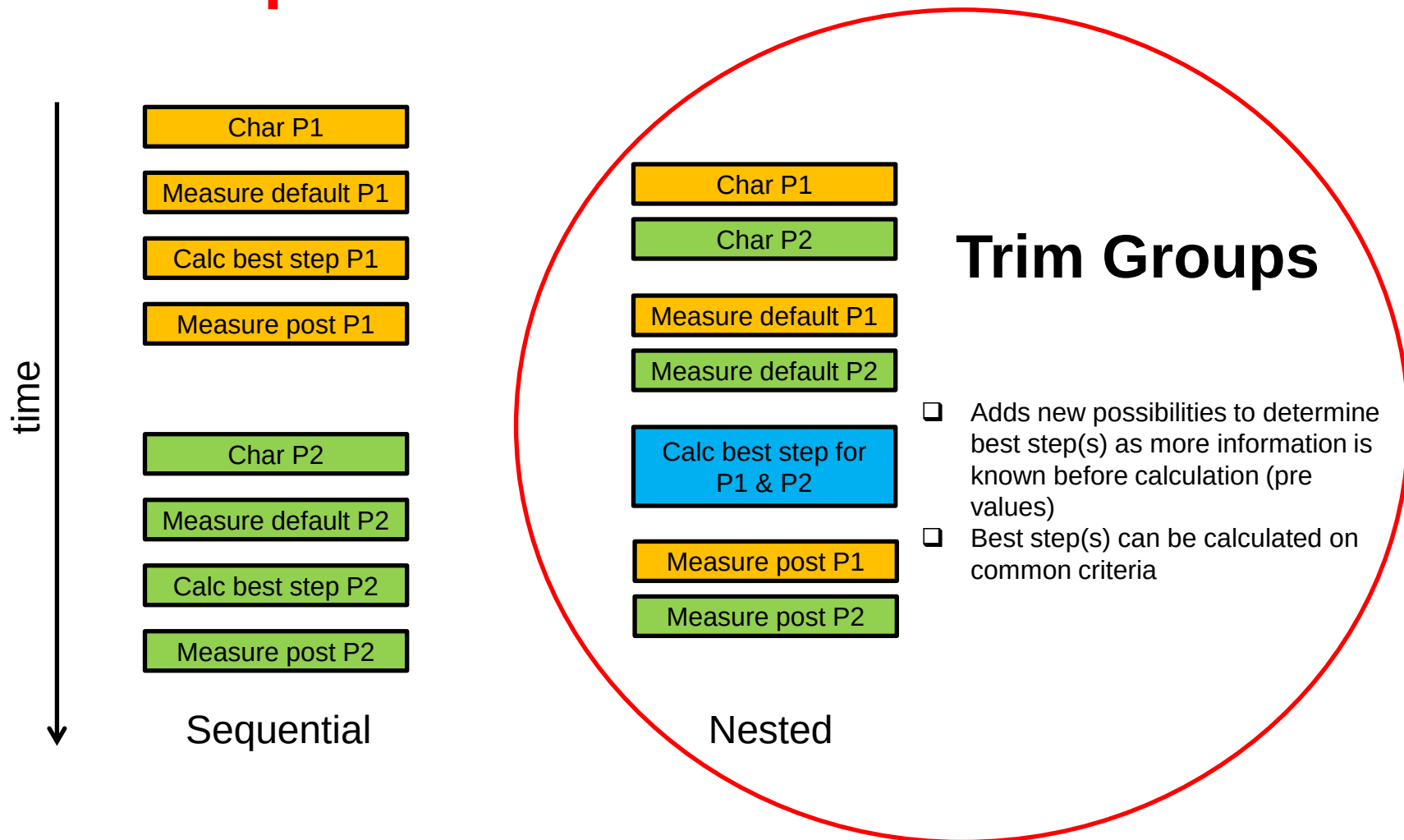
1102.1	-0.0800	UA	-0.5000	0.5000	ICON_abs_err *trim
1103.1	-1.6000	PCNT	-2.0000	2.0000	ICON_rel_err *trim

Datalogging

- TREG_LOG_DEBUG
 - for debugging
 - prints char table on ETS Shell screen

```
-----  
SITE 1  iconst  
-----  
          Result          Table    Delta abs    Delta rel  
step  0:    4.39e-006    -14.34%    +0.0000    +0.00%  
step  1:    4.53e-006    -11.51%    +1.45e-007    +3.30%  
step  2:    4.73e-006     -7.71%    +1.95e-007    +4.30%  
step  3:    4.92e-006     -4.00%    +1.90e-007    +4.02%  
step  4:    5.12e-006     +0.00%    +2.05e-007    +4.17%  
step  5:    5.35e-006     +4.39%    +2.25e-007    +4.39%  
step  6:    5.59e-006     +9.17%    +2.45e-007    +4.58%  
step  7:    5.86e-006    +14.44%    +2.70e-007    +4.83%  
-----
```

Trim Sequence



Px = Parameter x

Measure trim group (new)

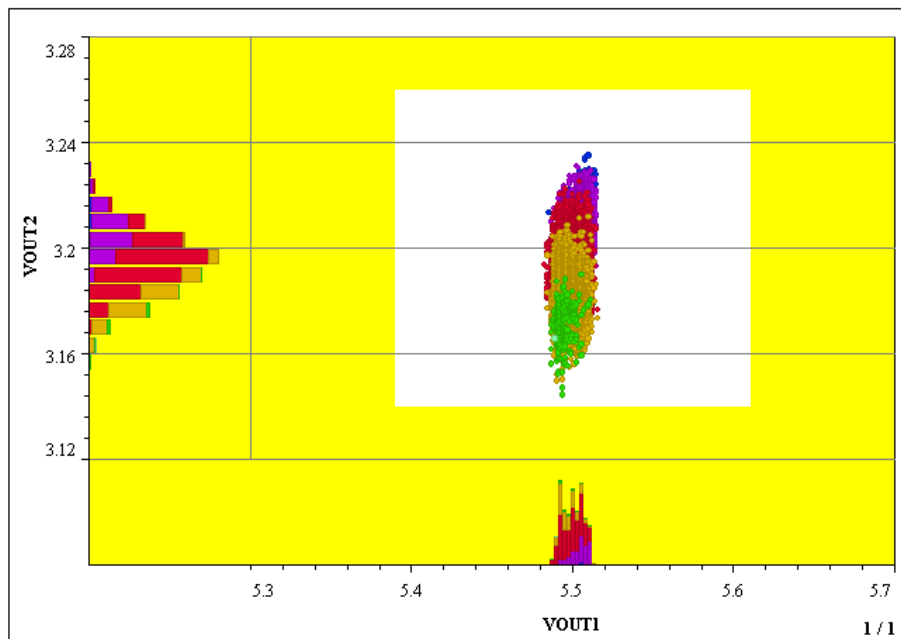
- **Multiple parameters can be trimmed individually but measured at the same time**
- Example: Multiple LDO outputs, independently trimmed, measured parallel
- Advantage: Test time benefit

Dependent trim group

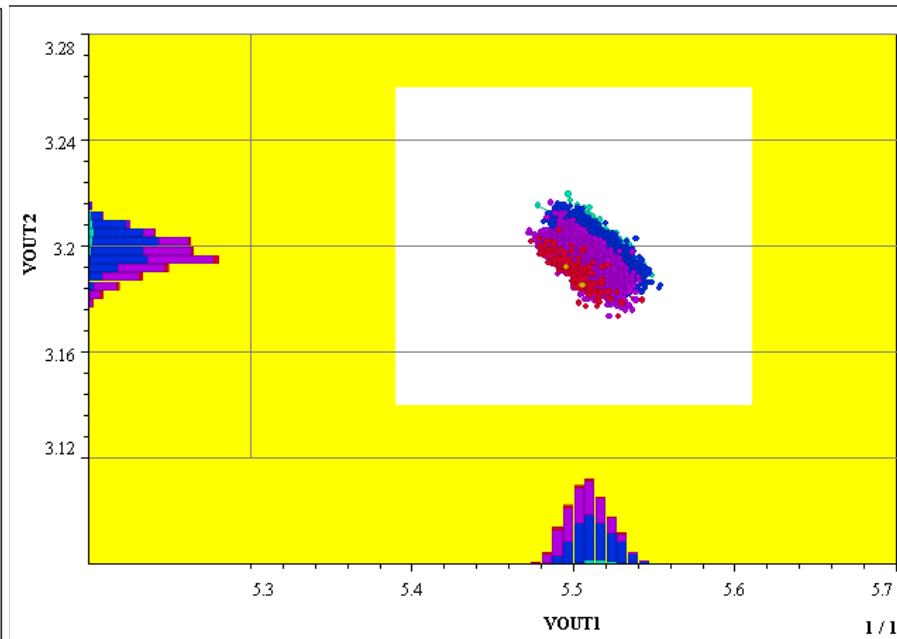
- **One trim influences multiple parameters**
- Example: Multiple VOULTs derived from band gap trim
- Traditional trim approach:
 - Trim one parameter and hope others are ok
 - Incremental (adjust again if it's not ok the first time)
- New approach:
 - Find best step at the first try
- Advantage:
 - Get the most out of the silicon (tighter specs, no yield loss)

Real world example: dependent trim

Standard approach:
Trim VOUT1 to 5.5V
Measure VOUT2



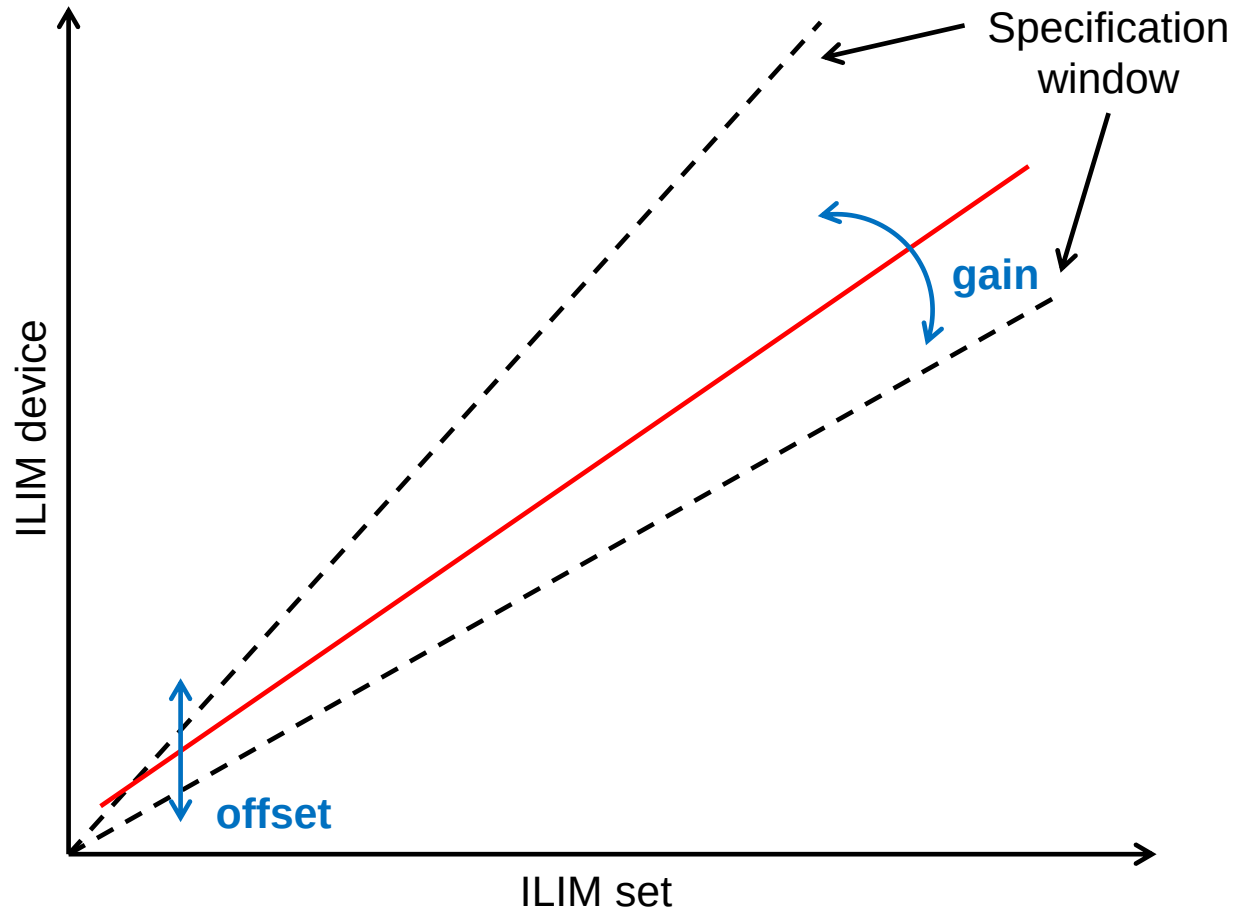
Trim Group approach:
Trim both best



In-dependent trim group

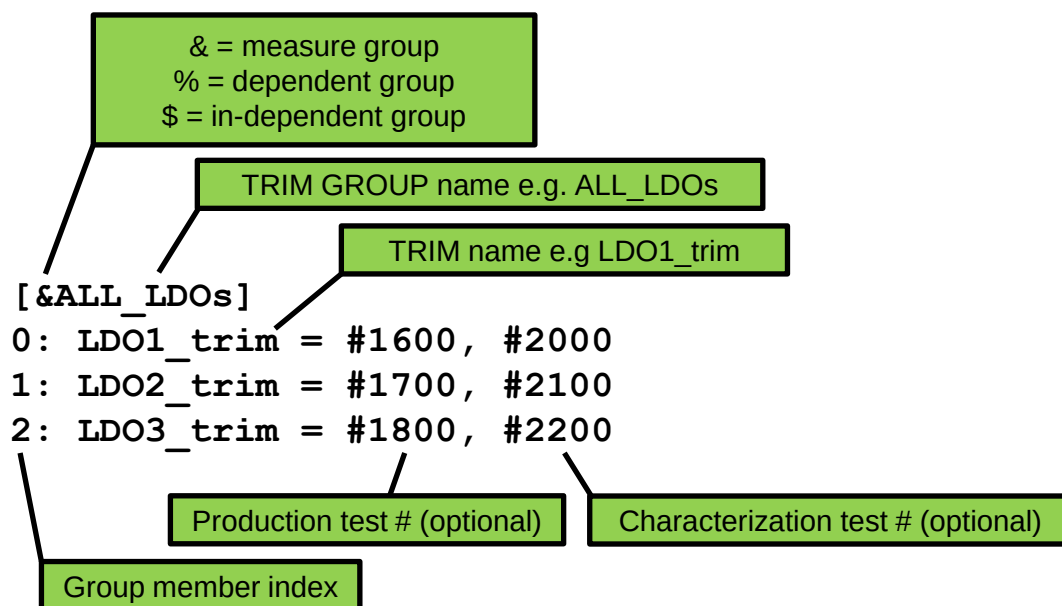
- **Multiple trims influencing multiple parameters**
- Example: Gain and offset trimming
- Traditional trim approach:
 - Trim one parameter and hope others are ok
 - Incremental (adjust again if it's not ok the first time)
- New approach:
 - Find best step at the first try by loop through all step combinations to find optimal step (using learned tables)
- Advantage:
 - Get the most out of the silicon (tighter specs, no yield loss)

Real world example: in-dependent trim



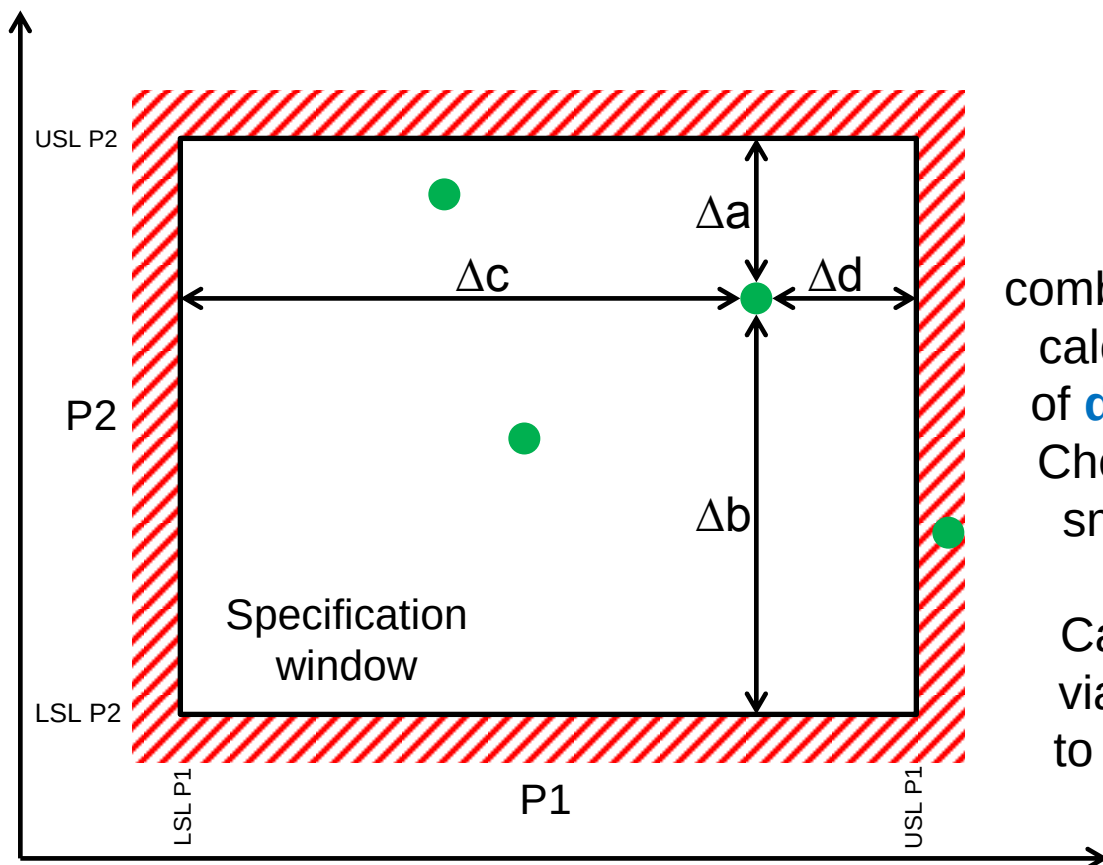
Trim group setup in *.treg file

- First setup trim parameters the same way as without using trim groups
- Setup trim groups as following



- set_test_num() method can be used at runtime to set production and char test number

How do we find best step?



● Projected post trim value

Algorithm

Go through all trim step combinations for all parameters and calculate **sum of least squares of deltas to specification limits**. Choose the combination with the smallest sum of least squares.

Calculation can be customized via a user defined function e.g. to weight one parameter higher

Implementation notes

- Prerequisites are the same as with execute() for single trim parameter
- Datalogging follows the same rules as for single parameter
- Use set_spec() method for every group member to set the specification window (=“target”)
- Or set them up in the *.treg file on each trim parameter using #<tnum>

```
[VOUT1]
Target = #1006
Table  = 1.62%,*1.31%,0.99%,0.67%
```

```
[VOUT2]
Target = #1056
Table  = 1.62%,*1.31%,0.99%,0.67%
```

```
[%VOUT]
0: VOUT1 = #1000, #10000
1: VOUT2 = #1050, #10016
```

Code example

```
// Test Function: vout_trim
ETS_PRGFLOW_FUNC vout_trim( int DSIndex, LPCTSTR TestLabel )
{
    TRIM_GRP_NODE &vout = dut.trim_grp("vout"); // shortcut

    // set specification window for VOUT1 and VOUT2
    vout(0).set_spec(1006); // get VOUT1 post trim limits(T#1006)
    vout(1).set_spec(1056); // get VOUT2 post trim limits(T#1008)

    vout(0).set_test_num(1000,10000); // test# prod=1000, test# char=10000
    vout(1).set_test_num(1050,10016); // test# prod=1050, test# char=10016

    // prepare DUT setup
    apuset(A_FB,APU_FV,0.0,APU_10V,APU_100MA,PIN_TO_VI);

    vout.execute(&measure_vout_trim);

    // cleanup DUT setup
    apuset(A_FB,APU_VIOFF,0.0,APU_10V,APU_100MA,PIN_TO_VI);

    return( msSiteStat( MS_ALL ) );
} // END_ETS_PRGFLOW_FUNC
```

Update & measure function - Notes

- execute() calls the update & measure function with the 'working' for each TRIM parameter set as needed.
- Update & measure function prototype has to be as following

```
void func_name(TRIM_GRP_NODE *trim_grp_node, TREG_MEASURE_FLAG treg_measure_flag,  
               TREG_RESULTS &meas_results);
```

- **trim_grp_node** pointer can be used to access TREG internals if needed
- **treg_measure_flag** is available to determine at which stage the function was called (pre, post or during char)
- **meas_results** is a two-dimensional array and has to be filled with results using

meas_results[group member index][site]

Update & measure function

```
void measure_vout_trim(TRIM_GRP_NODE *trim_grp_node, TREG_MEASURE_FLAG treg_measure_flag,
                      TREG_RESULTS &meas_results){

    double vout_1_res[NUM_SITES]; double vout_2_res[NUM_SITES];

    // update DUT registers with values from 'working'
    es.write_reg("LAHEAD");

    // VOUT1 measurement
    dpinforce(EN2, "1R", 3.6, 0.0, MS_ALL); // set EN2 to high -> VOUT1
    apuset(A_VOUT,APU_FVAWG,vout1_pat(0).sourced(1),APU_10V,vout1_pat(0).irange(),PIN_TO_VI);
    run_apat_and_capt_w_dpat(vout1_pat,"vout_cap",EN1,&vout_1_res);

    // VOUT2 measurement
    dpinforce(EN2, "0R", 3.6, 0.0, MS_ALL); // set EN2 to low -> VOUT2
    apuset(A_VOUT,APU_FVAWG,vout2_pat(0).sourced(1),APU_10V,vout2_pat(0).irange(),PIN_TO_VI);
    run_apat_and_capt_w_dpat(vout2_pat,"vref_cap",EN1,&vout_2_res);
    dpinforce(EN2, "1R", 3.6, 0.0, MS_ALL); // set EN2 back to VOUT high

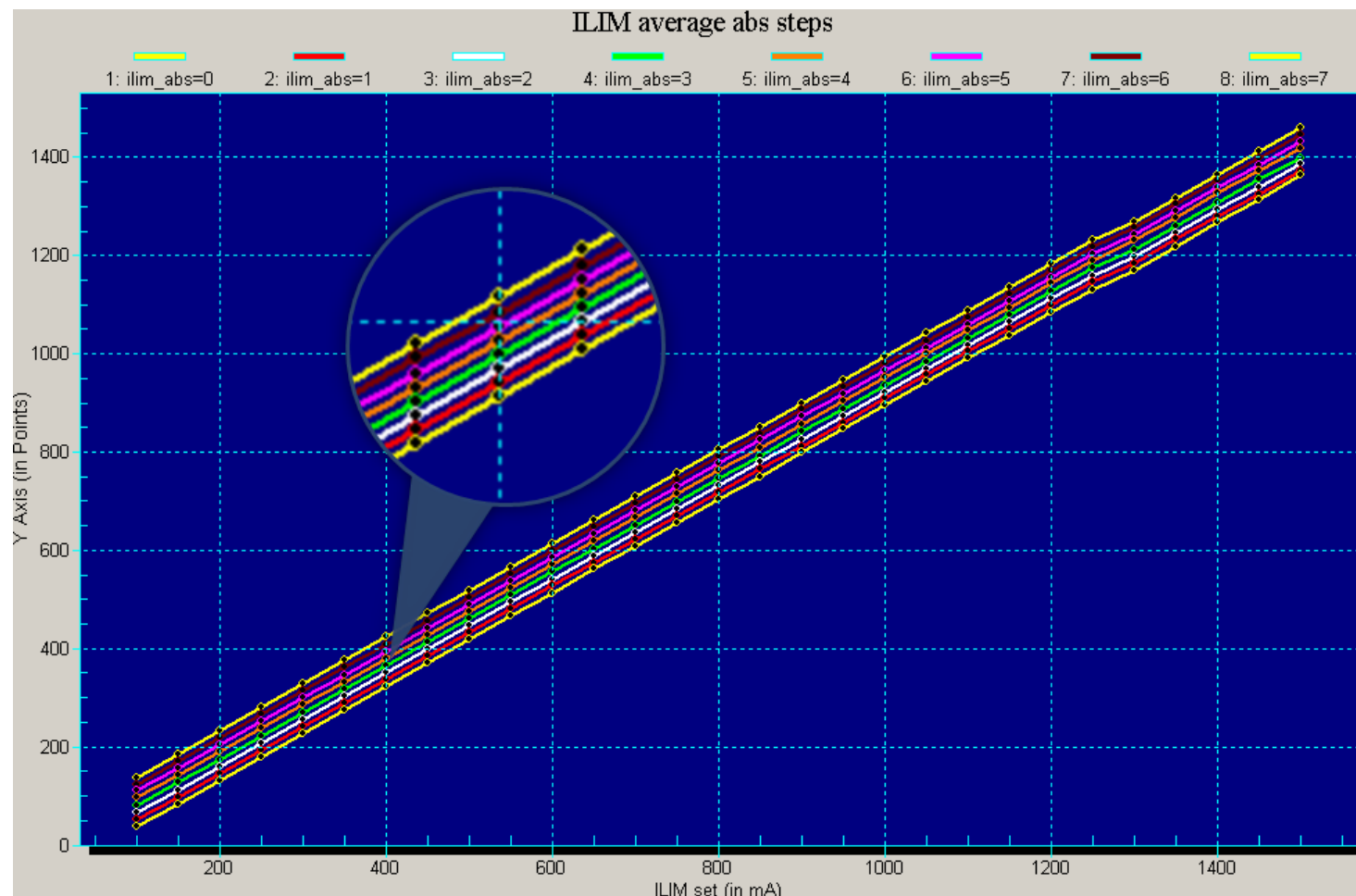
    // feedback results to TREG
    // meas_results[group member index][site]
    SERIAL meas_results[0][SITE] = vout_1_res[SITE];
    SERIAL meas_results[1][SITE] = vout_2_res[SITE];
}
```

Questions?

Thank you!

Backup

Current limit offset trim steps



Current limit gain trim steps

